

Lab-3 Assignment

Procedure

Accept the assignment

Submit all your code to your repository under Lab 3 folder by the due date time.

Procedure

Project Setup

- Create a new Lab3 subdirectory inside your repository on your local machine.
- Start the STM32CubeIDE software and create a new STM32 project.
- Search and Select our Nucleo-64 board.
- Initiate all peripherals with their default mode.
- Enable the USART2 global interrupt.

Create the new CLI interface

Using the information in the videos and handout, create a CLI that has a status window that displays the status information for the board. It is up to you how you want to provide that information to the user but of course the cleaner and more concise the better. The command scroll window should appear below this window and provide a command prompt for the user to submit requests. This status window should be updated if a command comes in that changes the status of the board.

USART

Change the way in which you receive characters from the user. Make sure you are using the non-blocking method.

Timer and Blinky

Previously you implemented a timer to toggle the state of the LED. Bring that code into your main loop and whatever the state of the LED is you should update the status window.

Project Objectives

Your command-line interface should provide:

- A status window and a command window created with ANSI escape sequences.
- A scrolling region within the status window.
- Interrupt-driven USART reception, one character at a time.
- Character echo and command-line editing.
- A timer that toggles an LED every two seconds.
- Status updates when the timer expires or a command changes the LED.

Advised Software Architecture

Divide the application into three areas of responsibility.

USART receive interrupt

The USART receive-complete callback should:

- Accept one received character and echo this to the console.
- Store the character in a receive queue ...array maybe.
- Once a carriage return is detected then you will indicate a command is ready for processing.
- Immediately restart interrupt-driven reception for the next character.
- Return promptly.

Reception must be restarted inside the callback. Otherwise, the program will receive the first character but no later characters.

Timer interrupt

The timer callback should:

- Toggle the LED every two seconds.
- Record that the timer caused the change.
- Request a status-window update.
- Return promptly.

It should not format strings, transmit USART messages, parse commands, or redraw the terminal.

Main loop

The main loop should:

- Detect commands are ready and process them.
- Check for timer and command events.
- Update the status window.
- Perform all terminal output.

This arrangement prevents interrupt callbacks from competing for the USART transmitter.

Suggested terminal layout

A useful 24-row layout is:

```
Status and Events
LED toggled by timer: ON
Command changed LED: OFF
Timer expired: LED ON
...
```

```
-----
Command responses appear here
-----
```

```
> led on_
```

The command window should scroll.

Submission

Make sure you have committed your latest solution and pushed it to your repository for grading.